DAT22-3

Software Engineering & Code Design

The terminal, Git, and object-oriented design for collaborative data work

---

How you structure the repository and how you structure the code inside it are the same question.

Albert School — 28 hours

Preface

In your first programming course you learned to make a computer do what you mean: core types, control flow, functions, files, a first class, a first API call. You could already write a script that reads data, transforms it, and writes a result. That script worked — on your machine, run by you, today.

This course is about the gap between **a script that works** and **software a team can trust**. It has two halves that look unrelated and are in fact one subject. The first half is professional infrastructure: the terminal you drive instead of the mouse, Git for keeping and sharing every version of your work, virtual environments and a project layout that let a teammate clone your repository and reproduce your results exactly. The second half is code design: how to shape classes so that a growing program stays readable — inheritance and composition, encapsulation, dunder methods, refactoring, and the automated tests that stop a fix in one place from breaking another.

We teach them together on purpose. The question "how do I structure this repository?" and the question "how do I structure the code inside it?" are inseparable in practice: a clean repository full of tangled code is no better than a tidy class buried in a chaotic folder. Both are answers to the same underlying need — work that other people (your future self included) can read, change, and rely on.

A warning about how to read this book. The first half cannot be learned by reading; it can only be learned by typing. Have a terminal open, a real folder to experiment in, and a throwaway GitHub repository to break. Every command in these pages is meant to be run, not admired. From S3 onward every course assumes you have these reflexes in your fingers — so spend the keystrokes now.

Contents

1 The terminal

You have been running scripts from a terminal since your first course, but mostly as a place to type `python script.py`. Now we make the terminal itself a tool. The **shell** — the program that reads what you type and runs it — is the most direct way to drive a computer, and the only practical way to drive the remote servers where real data work happens, which have no screen to click on. Its commands are terse because you type them all day; the terseness is a feature once your fingers learn it.

We assume a **Bash**-like shell, the default on Linux and on macOS (and available on Windows through Git Bash or WSL).

1.1 Where am I, and what is here?

A shell session always sits in one folder, the **working directory**. Three commands orient you:

```
pwd          # "print working directory" — the folder you are in
ls           # "list" — the files and folders here
ls -l        # the long form: permissions, size, modification date
ls -la       # also show hidden files (names starting with a dot)
```

The dot-files matter to us: `.git`, `.venv`, and `.gitignore` all begin with a dot precisely so they stay out of an ordinary `ls`. When something seems missing, `ls -la` is the first reflex.

1.2 Moving around: `cd`

`cd` ("change directory") moves you. Paths come in two flavours, and confusing them is the commonest early stumble:

```
cd project          # relative: a folder named "project" inside here
cd /home/ada/work   # absolute: the full path from the root /
cd ..               # up one level, to the parent folder
cd ~                # home directory (also plain `cd`)
cd -                # back to the previous directory
```

Remark An **absolute path** starts at the filesystem root `/` and means the same thing no matter where you are standing. A **relative path** is read from your current directory. `..` always means "the parent", and you can chain it: `cd ../../data` goes up twice and into `data`. Picture the filesystem as a tree and these are just directions for walking it.

1.3 Finding things: `find`

When you do not know where a file is, search the tree with `find`:

```
find . -name "*.csv"          # every .csv at or below here ('.' = here)
find . -name "students.csv"   # a specific file, anywhere below
find . -type d -name "data"   # only directories (-type d) named data
```

`find` walks every folder under the starting point, so it sees what `ls` (which looks only in one folder) cannot.

1.4 Creating, copying, moving, deleting

These five verbs cover almost all file manipulation:

```
mkdir reports             # make a directory
mkdir -p data/raw/2024    # make a whole chain of nested directories
touch notes.txt           # create an empty file (or update its timestamp)
cp notes.txt backup.txt   # copy
cp -r data data_backup    # copy a folder and everything in it (-r = recursive)
mv notes.txt docs/        # move into the docs folder
mv old.txt new.txt        # rename (move to a new name in the same place)
rm backup.txt             # delete a file
rm -r data_backup         # delete a folder and its contents
```

Common pitfall `rm` does **not** move a file to a trash can — it deletes it, permanently and silently. There is no undo. `rm -r` on the wrong folder, or the infamous `rm -rf /`, can erase an entire project or system in a second. Before any `rm`, read the line you typed once more, and prefer to `ls` the target first to confirm it is what you think it is.

1.5 Reading and slicing text

A huge fraction of data work is looking at text files without opening an editor. Four commands and one operator do most of it.

`cat` prints a whole file; `head` and `tail` print just the start or end — vital when a data file has a million rows and you only want to see its shape:

```
cat students.csv          # print the entire file
head students.csv         # first 10 lines (the header and a peek at the data)
head -n 3 students.csv    # first 3 lines
tail -n 5 students.csv    # last 5 lines
tail -n +2 students.csv   # everything FROM line 2 on (skip the header)
```

`grep` searches for lines matching a pattern — the single most useful text tool you will own:

```
grep "Paris" students.csv        # lines containing "Paris"
grep -i "paris" students.csv     # case-insensitive
grep -c "Paris" students.csv     # count matching lines instead of printing them
grep -v "Paris" students.csv     # invert: lines NOT containing "Paris"
grep -rn "TODO" src/             # recursively (-r) through a folder, with line
numbers (-n)
```

1.6 Redirection and pipes

By default a command prints to your screen. **Redirection** sends that output to a file instead; a **pipe** feeds it as input to another command. This is where the shell becomes a toolkit rather than a list of tools.

```
ls -l > listing.txt          # '>'  write output to a file (replacing it)
echo "done" >> log.txt       # '>>' append output to the end of a file
grep "Paris" students.csv > paris.csv   # save only the Paris rows
```

The pipe `|` connects the output of one command to the input of the next, letting you build a small assembly line:

```
cat students.csv | grep "Paris" | wc -l   # how many rows mention Paris?
head -n 100 big.csv | grep -c "error"     # errors in the first 100 lines
```

Common pitfall `>` overwrites its target without warning — `sort data > data` will **destroy** `data` before `sort` ever reads it. When in doubt, write to a new file name and rename afterwards. Remember the pair: `>` replaces, `>>` appends.

Symbol	Meaning
`>`	send output to a file, replacing its contents
`>>`	send output to a file, appending to the end
`|`	send output of the left command as input to the right command

Table 1: The three plumbing operators. `>` and `>>` end in a file; `|` ends in another command.

1.7 Bash scripting

When you run the same handful of commands repeatedly, save them in a file and run the file — a **Bash script**. This is the shell equivalent of writing a function: a repetitive task, named and reusable.

Create `backup.sh`:

```
#!/usr/bin/env bash
# Archive today's CSV outputs into a dated folder.

stamp=$(date +%Y-%m-%d)          # run `date`, capture its output into a variable
mkdir -p "backups/$stamp"        # make the destination (quotes guard spaces)
cp outputs/*.csv "backups/$stamp/"
echo "Backed up $(ls outputs/*.csv | wc -l) files to backups/$stamp"
```

Three details make this a script and not just lines in a file:

- The first line, the **shebang** `#!/usr/bin/env bash`, tells the system which interpreter should run the file.
- A variable is set with `name=value` (**no spaces around** `=`) and read back with `$name`. `$(...)` runs a command and substitutes its output.
- Before you can run it, you must mark the file executable:

```
chmod +x backup.sh    # grant "execute" permission
./backup.sh           # run it (./ = "the file here", not a system command)
```

Common pitfall Spacing in Bash is unforgiving in a way Python is not. `stamp = $(date)` is wrong — the spaces make Bash try to **run a program called `stamp`**. It must be `stamp=$(date)`. And always quote variables that might contain spaces: `cp $file dest` breaks on a file named `my report.csv`, while `cp "$file" dest` does not.

Remark A script earns its keep by being **idempotent** where possible — safe to run twice. `mkdir -p` does not complain if the folder already exists, which is why we prefer it to plain `mkdir` inside scripts. Small choices like this are what separate a script you trust from one you babysit.

2 Version control with Git

Imagine a folder full of `report_final.py`, `report_final_v2.py`, `report_final_REALLY.py`. Everyone has lived this; it is version control done by hand, and badly. **Git** replaces it with a single folder that remembers **every** version of itself, who changed what and why, and lets several people work on the same files without overwriting each other. It is the backbone of all collaborative software, and non-negotiable in professional data work.

2.1 The mental model

A Git **repository** is an ordinary folder plus a hidden `.git` subfolder that records its entire history. The history is a sequence of **commits**, each a labelled snapshot of all the tracked files at one moment. You do not commit changes directly; a file passes through three places:

Place	What it holds
working directory	your files as they are right now, edited
staging area (the "index")	the changes you have selected for the next commit
repository (`.git`)	the permanent history of committed snapshots

Table 2: Git's three areas. `git add` moves changes from the working directory to the staging area; `git commit` moves the staged changes into permanent history.

This two-step — stage, then commit — feels like ceremony at first but is the source of Git's precision: you choose **exactly** which changes belong together in one commit, rather than being forced to save everything at once.

2.2 Starting a repository and making commits

```
git init                # turn the current folder into a repository
git status              # the command you will run more than any other
git add students.csv    # stage one file
git add .               # stage every change in the folder
git commit -m "Add raw student data"   # record the staged changes
git log --oneline       # the history, one commit per line
```

`git status` is your constant companion: it tells you what is changed, what is staged, and what to type next. Run it before and after everything until the rhythm is automatic.

A commit needs a **message**, and the message is not a formality — it is the note your teammates and your future self read to understand **why** a change was made.

Common pitfall Write commit messages as imperative summaries of **why**, not **what**. Good: `"Fix score parsing for rows with empty marks"`. Useless: `"update"`, `"stuff"`, `"fix"`. The diff already shows **what** changed line by line; the message must add the reason, which the diff can never show. A history of `"update"` commits is a history you cannot navigate.

2.3 Connecting to GitHub

So far the repository lives only on your machine. **GitHub** hosts a copy on the internet — a **remote** — so you can back it up and share it. After creating an empty repository on GitHub:

```
git remote add origin https://github.com/you/project.git  # name the remote "origin"
git push -u origin main     # upload your commits; -u links your branch to it
# ...later, after more commits...
git push                    # send new commits up
git pull                    # bring teammates' new commits down
```

`push` sends your local history up; `pull` brings others' history down. The cycle of a working day is **pull, work, commit, push**.

2.4 Branching

A **branch** is an independent line of development — a way to work on a new feature without disturbing the known-good `main` branch. You start one, commit freely on it, and only fold it back into `main` when it is ready.

```
git switch -c add-scraper   # create branch "add-scraper" and move onto it
# ... edit files, git add, git commit, repeat ...
git switch main             # go back to main
git merge add-scraper       # fold the branch's commits into main
git branch -d add-scraper   # delete the finished branch
```

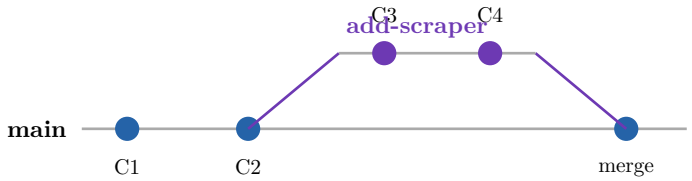

Figure 1: Branching. Work happens on `add-scraper` (commits C3, C4) while `main` stays stable; the merge brings that work back in as a single integration point.

This is the mechanism that lets a team divide work: each person takes a branch, and `main` is never broken by half-finished work.

2.5 Pull requests and code review

On a shared project you usually do not merge your own branch straight into `main`. Instead you open a **pull request** (PR) on GitHub: a proposal to merge your branch, where teammates can read the diff, comment line by line, and approve before it lands. The workflow:

1. Push your branch: `git push -u origin add-scraper`.
2. On GitHub, open a pull request from `add-scraper` into `main`.
3. A teammate reviews it, leaving comments — questions, suggestions, requested changes — anchored to specific lines.
4. You address the feedback with more commits and push again; the PR updates automatically.
5. Once approved, you **merge** the PR, and the branch's work joins `main`.

A good review comment is specific and kind, and distinguishes a blocking problem from a preference. "This will divide by zero when `scores` is empty — guard it?" is worth more than "looks good" or "I'd have done this differently." The point of review is not gatekeeping; it is that a second pair of eyes catches what the author, too close to the code, cannot see.

2.6 Merge conflicts

When two branches change the **same lines** of the same file, Git cannot decide which version wins and stops to ask you. This is a **merge conflict**, and beginners fear it far more than they should. Git marks the clash directly in the file:

```
<<<<<<< HEAD
PASS_MARK = 60
=======
PASS_MARK = 50
>>>>>>> add-scraper
```

The block between `<<<<<<<` and `=======` is the version already on your current branch (`HEAD`); the block between `=======` and `>>>>>>>` is the version coming from the branch you are merging. To resolve it, you:

1. Open the file and find each conflict marker.
2. Decide the correct final text — keep one side, the other, or a blend.
3. Delete all three marker lines (`<<<<<<<`, `=======`, `>>>>>>>`), leaving only the resolved content.
4. Stage and commit: `git add` the file, then `git commit` to complete the merge.

```
# resolved by hand to:
PASS_MARK = 60
```

> **Common pitfall** The conflict is not resolved until **every** marker is gone and you have committed. A frequent disaster is committing a file with a `<<<<<<<` still in it — now your code contains literal garbage and will not run. Before staging a resolved file, search it for the marker (`grep -n "<<<<<<<" file.py`) to be sure none remain. Resolving a conflict is a decision about meaning, not a mechanical merge — read both sides and understand why they differ before choosing.

3 Reproducible environments

A teammate clones your repository, runs your script, and it crashes with `ModuleNotFoundError`. The code is fine; their machine simply does not have the libraries yours does, or has different versions. The cure — which you met briefly in your first course — is to make the **environment** part of the project, captured in the repository and rebuildable on any machine.

3.1 Why isolation matters

A **virtual environment** is a project-local folder holding its own Python interpreter and its own installed packages. Two projects can then depend on two incompatible versions of the same library without colliding, because each looks only inside its own environment. It isolates **which packages, at which versions** a project sees — nothing else.

```
python -m venv .venv          # create the environment in a folder named .venv
source .venv/bin/activate     # activate it (macOS / Linux)
.venv\Scripts\activate        # activate it (Windows)
# your prompt now shows (.venv); pip now installs into this folder only
deactivate                    # step back out when done
```

3.2 Pinning dependencies with `requirements.txt`

Isolation only helps a teammate if you also record **what** to install. Inside the activated environment, freeze the exact versions into a file:

```
pip freeze > requirements.txt
```

```
pandas==2.2.0
requests==2.31.0
pytest==8.0.0
```

Now anyone — a teammate, a grader, or you on a new laptop — recreates the environment exactly:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt    # install precisely these versions
```

Common pitfall Never commit the `.venv` folder to Git. It is large, machine-specific, and fully rebuildable from `requirements.txt` — committing it bloats the repository and causes endless spurious conflicts. Add a line `.venv/` to a `.gitignore` file so Git ignores it. What you **do** commit is the `requirements.txt`: the recipe, not the meal.

Remark `requirements.txt` plus your committed code is the same **reproducibility** promise from your first course, now made to a whole team: clone this repository, build the environment from this file, and you will get the result I got. That promise is the entire point of the infrastructure half of this course.

4 Collaborative project structure

Where a file goes is itself a design decision. A repository that a newcomer can navigate without asking is one with a **convention**, and the data-science world has largely settled on one. Following it means anyone who has seen one such project can find their way around yours.

4.1 The standard layout

```
project/
├── data/          # input data — raw, read-only, never edited by hand
├── src/           # the source code: importable modules and classes
├── notebooks/     # exploratory Jupyter notebooks (messy by nature)
├── outputs/       # generated results: figures, processed files, reports
├── tests/         # automated tests (next chapter)
├── README.md      # what this project is and how to run it
├── requirements.txt
└── .gitignore     # files Git should ignore (.venv/, outputs/, data/, ...)
```

The separation is not bureaucracy; it encodes a discipline. `data/` is **raw and read-only** — you never overwrite an input, so you can always re-run from a known starting point. `outputs/` is **disposable** — anything in it can be regenerated by running the code, which is why it is usually git-ignored. `src/` holds the real, reusable logic; `notebooks/` holds the throwaway exploration that **uses** that logic. Keeping exploration out of `src/` is what stops a research repository from turning into a swamp.

4.2 The README as contract

The `README.md` is the first file anyone opens, and it is a **contract**: it promises what the project does and exactly how to make it run. A serviceable README answers four questions:

1. **What** is this project, in one or two sentences?
2. **How do I set it up?** — the exact commands: create the venv, install requirements.
3. **How do I run it?** — the command that produces the results.
4. **Where does what live?** — a one-line tour of the folders above.

> **Remark** The test of a README is brutal and simple: hand your repository to someone who has never seen it, and watch them follow only the README. Every place they get stuck or have to ask you is a gap in the contract. Markdown — the `.md` format — is the lightweight syntax (`#` for headings, fenced triple-backtick blocks for code) that GitHub renders into a formatted page; a few minutes learning it pays off on every project.

4.3 Dividing work without breaking `main`

The layout and Git's branches combine into a way of working as a team. Each person works on a **branch** (the previous chapter), touching mostly their own files under `src/`; `main` always holds a version that runs. Conflicts are rarest when work is partitioned so that two people seldom edit the same file — which good module boundaries, the subject of the rest of this book, naturally encourage. Here the two halves of the course meet: **how you split the code into modules decides how cleanly you can split the work across people.**

5 Object-oriented design for data work

Your first course showed you how to **write** a class — `__init__`, `self`, attributes, methods. This chapter is about how to **design** with classes: choosing what should be a class at all, and how classes should relate. We work through the archetypes you actually meet in data projects — a loader, a scraper, a pipeline — because design is learned through cases, not slogans.

5.1 A class is a noun with responsibilities

The first design question is **what deserves to be a class**. A good rule: a class models a "thing" in your problem that bundles some data with the operations on that data, and that you will create several of, or want to swap out. In a data project, `DataLoader`, `Scraper`, `Cleaner`, `Pipeline`, `Report` are natural classes; a one-off arithmetic calculation is not. Start from the nouns of your problem.

```
class DataLoader:
    """Loads a dataset from a CSV file and reports basic facts about it."""

    def __init__(self, path: str):
        self.path = path
        self.rows: list[dict] = []

    def load(self) -> list[dict]:
        import csv
        with open(self.path) as f:
            self.rows = list(csv.DictReader(f))
        return self.rows
```

5.2 Inheritance: an "is-a" relationship

Inheritance lets one class build on another, taking its attributes and methods and adding or overriding some. It models an **is-a** relationship: a `CSVLoader` **is a** `DataLoader`, a `JSONLoader` **is a** `DataLoader`. The shared behaviour lives once in the parent; each child specialises only what differs.

```
class DataLoader:
    def __init__(self, path: str):
        self.path = path

    def load(self) -> list[dict]:
        raise NotImplementedError("subclasses must implement load()")

    def describe(self) -> str:
        return f"{self.__class__.__name__} reading {self.path}"

class CSVLoader(DataLoader):            # CSVLoader IS A DataLoader
    def load(self) -> list[dict]:
        import csv
        with open(self.path) as f:
            return list(csv.DictReader(f))

class JSONLoader(DataLoader):           # so is JSONLoader
    def load(self) -> list[dict]:
        import json
        with open(self.path) as f:
            return json.load(f)
```

Both children inherit `__init__` and `describe` from `DataLoader` and supply their own `load`. Code that has a `DataLoader` can call `.load()` and `.describe()` without caring which kind it holds — the parent defines a **contract** the children honour.

5.3 Composition: a "has-a" relationship

Composition builds a class out of **other objects it holds as attributes**. It models a **has-a** relationship: a `Pipeline` **has a** loader, **has a** cleaner, **has a** writer. Rather than inheriting their behaviour, it delegates to them.

```
class Pipeline:
    """Runs a data job by delegating to the objects it is built from."""

    def __init__(self, loader: DataLoader, cleaner: "Cleaner"):
        self.loader = loader     # HAS A loader
        self.cleaner = cleaner   # HAS A cleaner

    def run(self) -> list[dict]:
        rows = self.loader.load()
        return self.cleaner.clean(rows)
```

The `Pipeline` does not **become** a loader; it **uses** one. And because it holds the loader as a parameter, you can hand it a `CSVLoader` today and a `JSONLoader` tomorrow without touching `Pipeline` at all.

5.4 Choosing between them

This is the design decision the OP asks you to justify, so meet it head-on.

Question	Inheritance	Composition
Relationship	"is-a"	"has-a"
`CSVLoader` and `DataLoader`	yes — a CSV loader **is a** loader	no
`Pipeline` and its loader	no	yes — a pipeline **has a** loader
Reuse by	specialising a parent	assembling parts
Flexibility	fixed at class-definition time	swappable at run time

Table 3: Inheritance models **is-a** and specialises a single type; composition models **has-a** and assembles independent parts.

The practical guidance, in one line: **use inheritance when a class is a special kind of another class; use composition when a class is built from several others.**

Common pitfall The classic mistake is reaching for inheritance whenever you want to reuse code. If `Pipeline` inherited from `CSVLoader` just to call its `load`, you would be claiming a pipeline **is a** CSV loader — which is false, and which locks the pipeline to CSV forever. When the relationship is "I need what that object can do", that is **has-a**: hold the object, do not inherit from it. Seasoned designers **prefer composition by default** and reach for inheritance only when the is-a is genuine.

6 Encapsulation and dunder methods

A class is a promise: "use me through these methods, and do not worry about how I store things inside." **Encapsulation** is the discipline of keeping that promise — separating the **public contract** (what callers may rely on) from the **private internals** (what you are free to change). Done well, you can rewrite the inside of a class without breaking a single line that uses it.

6.1 Public contract, private internals

Python has no enforced "private", but it has a firm **convention**: an attribute whose name starts with an underscore is private — an implementation detail nobody outside should touch.

```
class Dataset:
    def __init__(self, rows: list[dict]):
        self._rows = rows          # private: how we store the data

    def add(self, row: dict) -> None:
        """Public contract: add a record."""
        self._rows.append(row)

    def count(self) -> int:
        """Public contract: how many records."""
        return len(self._rows)
```

Callers use `add` and `count`; they are not meant to reach in and mutate `_rows`. The leading underscore is you saying "touch this and you are on your own." This freedom — to change `_rows` from a list to something else later, as long as `add` and `count` keep working — is the whole payoff of encapsulation.

Common pitfall Encapsulation is not about secrecy; it is about **being able to change your mind**. Code that reaches into another object's `_private` attributes welds the two

together: now you cannot alter the internals without hunting down every reacher. Expose a deliberate set of public methods and route all access through them, even your own.

6.2 Dunder methods: behaving like a built-in

You have called `len([1,2,3])`, printed objects, compared them with `==`. These work because the types implement special methods with double-underscore names — **dunder** methods. Implement them on **your** class and your objects gain the same fluency, behaving like Python's built-ins instead of opaque blobs.

```
class Dataset:
    def __init__(self, rows: list[dict]):
        self._rows = rows

    def __len__(self) -> int:
        """Now len(dataset) works."""
        return len(self._rows)

    def __str__(self) -> str:
        """Now print(dataset) is readable, not <Dataset object at 0x...>."""
        return f"Dataset with {len(self._rows)} rows"

    def __eq__(self, other) -> bool:
        """Now dataset1 == dataset2 compares contents, not identity."""
        if not isinstance(other, Dataset):
            return NotImplemented
        return self._rows == other._rows
```

With these in place your object speaks Python's native idioms:

```
d1 = Dataset([{"a": 1}])
d2 = Dataset([{"a": 1}])
len(d1)        # 1              -> __len__
print(d1)      # Dataset with 1 rows   -> __str__
d1 == d2       # True           -> __eq__ compares contents
```

Dunder	Triggered by	Should return
`__init__`	`Dataset(...)`	nothing — it sets up `self`
`__str__`	`print(d)`, `str(d)`	a human-readable string
`__len__`	`len(d)`	a non-negative `int`
`__eq__`	`d1 == d2`	`True` / `False` (or `NotImplemented`)

Table 4: Common dunder methods and what triggers each. Implementing them makes custom objects behave like Python built-ins.

Common pitfall `__eq__` has two traps. First, without it, `==` falls back to **identity** — `d1 == d2` is `True` only if they are literally the same object, so two datasets with equal contents compare unequal. Second, when the other operand is of a different type, return the special value `NotImplemented` (not `False`) so Python can try the other object's `__eq__`; returning `False` outright can mask correct comparisons. Use `isinstance` to check the type before comparing.

7 Refactoring to objects

Refactoring is changing the **structure** of code without changing what it **does** — making it clearer, not different. The OP's exercise is the one every data scientist eventually faces: a procedural script that grew organically into a tangle, rewritten as a small set of well-chosen objects. We do it as a worked transformation so you can see each decision.

7.1 The procedural starting point

Here is a script that loads students, filters the passers, and prints an average. It works — and it is already hard to change or test.

```
import csv

rows = []
with open("students.csv") as f:
    for r in csv.DictReader(f):
        r["score"] = int(r["score"])
        rows.append(r)

passers = []
for r in rows:
    if r["score"] >= 60:
        passers.append(r)

total = 0
for r in passers:
    total += r["score"]
average = total / len(passers)
print(f"{len(passers)} passed, average {average:.1f}")
```

The smell: loading, filtering, and summarising are three distinct responsibilities mashed into one straight-line block. Nothing here can be reused or tested in isolation; change the pass mark and you must reread the whole script to be sure you have not broken something.

7.2 Step 1 — name the responsibilities

Before writing a class, name the **jobs** the script does. Three stand out: **load** data, **filter** it, **summarise** it. Those nouns — a loader, a filter rule, a summary — are candidate classes or methods. This naming step is the actual design work; the code that follows just records the decision.

7.3 Step 2 — model them as a class

A `StudentReport` **has a** path (composition with a plain value) and offers methods for each job, with the score parsing encapsulated:

```
import csv

class StudentReport:
    def __init__(self, path: str, pass_mark: int = 60):
        self._path = path
        self._pass_mark = pass_mark
        self._rows: list[dict] = []

    def load(self) -> "StudentReport":
        with open(self._path) as f:
            self._rows = [
                {**r, "score": int(r["score"])}
```

```
                for r in csv.DictReader(f)
            ]
        return self                     # return self so calls can chain

    def passers(self) -> list[dict]:
        return [r for r in self._rows if r["score"] >= self._pass_mark]

    def average_score(self) -> float:
        passers = self.passers()
        return sum(r["score"] for r in passers) / len(passers)

report = StudentReport("students.csv").load()
print(f"{len(report.passers())} passed, average {report.average_score():.1f}")
```

7.4 What the refactor bought, and what to justify

Every modeling decision should be defensible — the OP asks you to justify each:

- **Why a class and not three functions?** Because the three jobs share state (`_rows`, `_pass_mark`); a class bundles that state with the operations rather than passing it through every call.
- **Why `pass_mark` as a constructor argument?** So the threshold is configurable without editing code, and so two reports can use two thresholds at once.
- **Why private `_rows`?** So `load` can change how data is stored later without touching the public `passers`/`average_score` contract.

> **Common pitfall** Refactoring must be **behaviour-preserving**: the new code must produce the same results as the old. The only safe way to know is to have tests that pass before and after — which is exactly why refactoring and unit testing are taught together, and why the final chapter is the one that makes all the others safe to change. Refactor in small steps, running the tests after each, rather than rewriting everything and hoping.

8 Unit testing with pytest

Every chapter so far has assumed you can tell whether your code still works. **Unit tests** are how you know — automatically, in seconds, every time. A unit test is a small piece of code that runs a piece of your program and asserts that the result is what it should be. On a collaborative project they are indispensable: they are the safety net that lets anyone change code without fear of silently breaking someone else's work.

8.1 Why automated tests

You already test your code — by running it and eyeing the output. The problem is that manual testing does not **scale** and does not **persist**: you cannot re-check every feature by hand after every change, so regressions (a fix in one place that breaks another) slip in unnoticed. An automated test, written once, runs forever, and turns "I think it still works" into "the tests pass." It is the technical foundation of the pull-request review you met earlier: a reviewer trusts a branch partly because its tests are green.

8.2 A first test with pytest

pytest is the standard Python testing tool. A test is just a function whose name starts with `test_`, containing an `assert` — a statement that must be true:

```python
# src/stats.py
def average(numbers: list[float]) -> float:
    return sum(numbers) / len(numbers)
```

```python
# tests/test_stats.py
from src.stats import average

def test_average_of_two_numbers():
    assert average([10, 20]) == 15

def test_average_of_one_number():
    assert average([7]) == 7
```

Run every test by typing `pytest` in the project root:

```
pytest                 # discover and run all tests
pytest -v              # verbose: one line per test
```

pytest finds files named `test_*.py`, runs every `test_` function, and reports which assertions held. A green dot per passing test, an `F` and a precise explanation per failure — including the values it expected and got.

8.3 Testing for the errors you expect

Good tests check not only the happy path but the **edges**: empty input, bad input, the boundary cases. To assert that code **raises** an exception, use `pytest.raises`:

```python
import pytest
from src.stats import average

def test_average_of_empty_list_raises():
    with pytest.raises(ZeroDivisionError):
        average([])
```

This test **passes** when `average([])` raises `ZeroDivisionError` — you are asserting the failure happens, which documents and locks in how your code behaves at the edge.

8.4 Fixtures: shared setup

Many tests need the same starting object — a sample dataset, a temporary file. Repeating that setup in every test is noise. A **fixture** is a function, marked `@pytest.fixture`, that builds the thing once; any test that names it as a parameter receives it ready-made.

```python
import pytest
from src.report import StudentReport

@pytest.fixture
def sample_csv(tmp_path):
    """Write a tiny CSV to a temp file and return its path."""
    path = tmp_path / "students.csv"
    path.write_text("name,score\nAda,90\nBo,40\n")
    return str(path)

def test_passers_excludes_low_scores(sample_csv):
    report = StudentReport(sample_csv, pass_mark=60).load()
    passers = report.passers()
    assert len(passers) == 1
    assert passers[0]["name"] == "Ada"

def test_average_counts_only_passers(sample_csv):
```

```
    report = StudentReport(sample_csv, pass_mark=60).load()
    assert report.average_score() == 90
```

Both tests name `sample_csv` as a parameter, so pytest runs the fixture for each and hands in a fresh sample. (`tmp_path` is a fixture pytest provides for free: a unique temporary directory, cleaned up automatically — never write test files into your real project.)

Common pitfall Tests must be **independent**: each should pass on its own and in any order, with no test relying on another having run first. Sharing one mutable object across tests is the classic way to break this — one test changes it, the next sees the change and fails mysteriously. Fixtures guard against exactly this by rebuilding the object for every test. If your tests pass together but fail when run singly (or vice versa), hidden shared state is the cause.

Remark Write tests as you write the code, not as a chore at the end. Each time you add a method, add a test for it — including one for the edge case you are tempted to skip, because that edge is where bugs live. The discipline feels slow for a week and saves you from the first regression that would have cost an afternoon to find by hand.

Where you have arrived. You can now work the way professionals do. You drive the terminal — navigating, slicing text with `grep` and pipes, automating with Bash. You keep and share your work in Git: committing meaningfully, branching, reviewing through pull requests, and resolving conflicts by hand. You make your work reproducible with virtual environments and a pinned `requirements.txt`, laid out in a repository whose README is a contract any teammate can follow. And you design the code inside it deliberately — choosing inheritance for **is-a** and composition for **has-a**, encapsulating internals behind public contracts, giving objects built-in fluency through dunder methods, refactoring tangled scripts into justified objects, and protecting all of it with pytest so that change is safe. The repository and the code within it are now one well-structured whole — which is exactly what every course from here on assumes you can build.